OliveTrace

Rapport DevOps

Intégration continue et déploiement

Applications Web Avancées · 22 octobre 2026

Du travail en équipe à une application déployable.

OliveTrace relie les acteurs de la filière oléicole tunisienne. Ce rapport décrit les contrôles réellement exécutés, le déploiement Docker local et les étapes restant à valider en cloud.

244 tests par base · **75,2 %** de couverture globale · **2,0 %** de duplication

CI VALIDÉE | DOCKER LOCAL VALIDÉ | CD PUBLIC NON VALIDÉ

Sofiene Zayati	Producteurs et fermes
Mariem Ammari	Récolte et moulin
Oussema Benjemia	Laboratoire et certification
Hana Fkiri	Distribution
Aymen Jenhani	Consommateurs

SOMMAIRE — Numéro de section · page du document

01 Processus et Git p. 2
02 Pipeline CI/CD p. 3
03 Preuves CI p. 4
04 Couverture des tests p. 5
05 Qualité SonarQube p. 6
06 Architecture Docker p. 7
07 Déploiement local p. 8
08 Reproduire en local p. 9
09 CD et Railway p. 10
10 Exploitation p. 11
11 Bilan et validation p. 12

Référence des tests et mesures : **bc96add**, branche **main**. Analyse SonarQube : 22 octobre 2026, 11 h 56 (Tunis). Les captures constituent des preuves datées.

01 Du développement à la livraison

Les contrôles CI sont automatisés. Le déploiement local est réalisé ; le déploiement public reste une étape à finaliser.

1 · Développer et intégrer avec Git

Branche feature/* → Pull Request → intégration dans develop → promotion vers main après revue. Les commits conservent l'historique du travail des cinq membres.

2 · Exécuter la CI GitHub Actions

Job qualité : dépendances → build Vite → Pint → tests SQLite et MySQL → SonarQube / Quality Gate.
Job dépendant : build Docker → migrations → démarrage → contrôles HTTP.

Un échec du job qualité empêche le job Docker.

3a · Déploiement local

Réalisé et vérifié.
Commande manuelle Docker Compose ; application + MySQL ; accès sur :8080.

3b · Déploiement public

Préparé, non validé.
Railway : liaison main / attente CI, services, variables et HTTPS à finaliser.

Lire le diagramme

Les traits pleins décrivent les étapes démontrées. Les pointillés indiquent la suite prévue en cloud. Le job Docker en CI vérifie un conteneur temporaire ; il ne publie pas automatiquement un site accessible en ligne.

Déclencheurs réels : PR vers main/develop, push sur ces branches et lancement manuel. La revue et la promotion de branche restent des actions humaines.

Preuves : historique Git · PR DevOps #8 · workflow figé au commit mesuré

02 Le pipeline CI, étape par étape

Configuration : .github/workflows/checks.yml. Les deux jobs s'exécutent sur des runners Ubuntu GitHub Actions.

Étape	Exécution	Résultat attendu
1 · Installer	PHP 8.4, Composer 2, Node 24 ; dépendances à partir des fichiers lock.	Manifeste et extensions compatibles.
2 · Préparer	MySQL 8.4 jetable ; migrations et seeders dans la CI.	Base distincte des données de démonstration.
3 · Construire	npm ci, build Vite et contrôle PHP avec Pint.	Assets compilés et formatage conforme.
4 · Tester	Suite exécutée séparément sur SQLite puis MySQL.	244 tests / 1 762 assertions par base.
5 · Analyser	Rapport Clover importé par SonarQube ; attente du gate.	Gate vert sous ses conditions configurées.
6 · Conteneur	Build Docker, migration, démarrage et contrôles HTTP.	Santé et pages accessibles ; chemins privés refusés.

Dépendance bloquante

Tests and code quality → Docker build and HTTP smoke check. Le second job déclare **needs: check**. Un Quality Gate en échec fait échouer le premier job.

Rapports et commandes de test

```
php vendor/bin/phpunit --coverage-clover=build/coverage.xml
php scripts/test-mysql.php
```

La CI produit aussi **build/junit.xml**. Clover et JUnit sont conservés 14 jours dans les artefacts. Les appels IA et e-mail sont simulés ; la disponibilité réelle de ces fournisseurs est testée séparément.

03 La preuve des contrôles automatisés

Une seule référence mesurée : bc96add. Les tests, la qualité et l'image de ce commit ont été vérifiés ensemble.

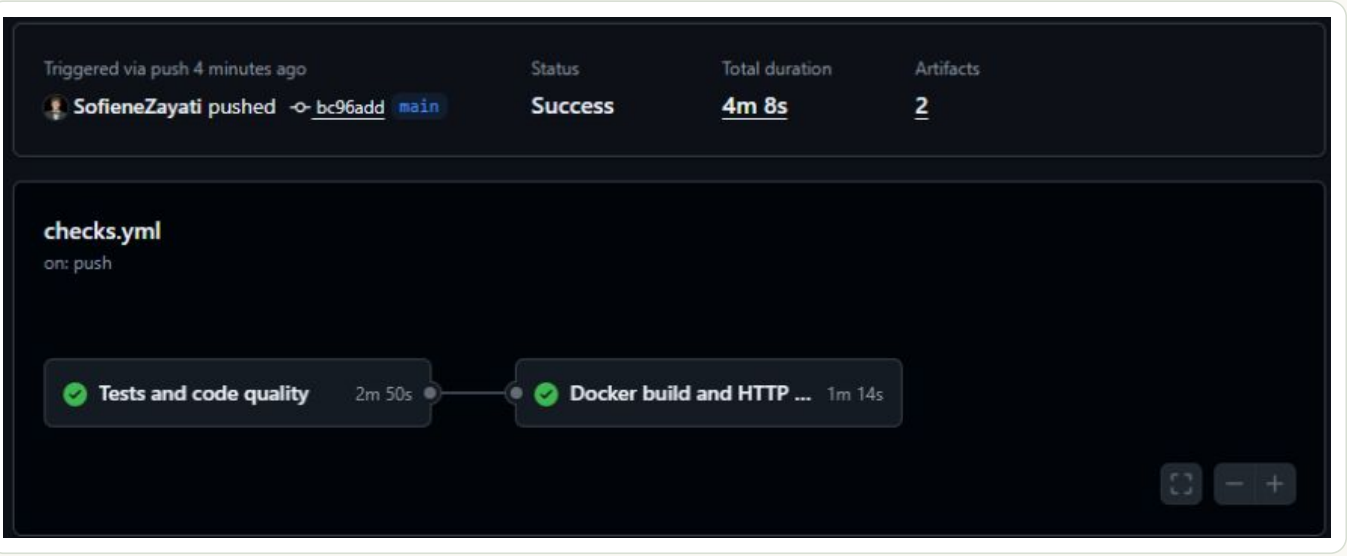


Figure 1. Exécution #40 sur main : statut **Success**, commit **bc96add**, deux jobs réussis. Durée totale : **4 min 8 s**.

Ouvrir l'exécution complète et ses artefacts

Résultats relevés dans les logs de cette exécution

Contrôle	Résultat vérifié
Suite SQLite	244 tests / 1 762 assertions ; aucun échec.
Suite MySQL	244 tests / 1 762 assertions ; aucun échec.
Qualité et frontend	Composer / extensions, build Vite, Pint et Quality Gate réussis.
Conteneur	Build, migration et contrôles HTTP réussis.

Ce que ces preuves permettent de conclure

Le même code passe les contrôles sur deux moteurs de test et peut démarrer dans son conteneur. Ces résultats ne prouvent ni un déploiement public, ni une disponibilité permanente des API IA.

Les écrans SonarQube sont évolutifs ; les captures et chiffres de ce rapport sont figés au commit ci-dessus. Le lien GitHub Actions conserve les étapes et les rapports associés.

04 Couverture : mesure et périmètre

La couverture mesure les lignes exécutées pendant les tests. Elle ne mesure pas, à elle seule, la correction fonctionnelle.

75,2 %

Couverture globale mesurée

5 142 lignes exécutées sur **6 841** lignes mesurables.

Exécutées : 5 142 Non exécutées : 1 699

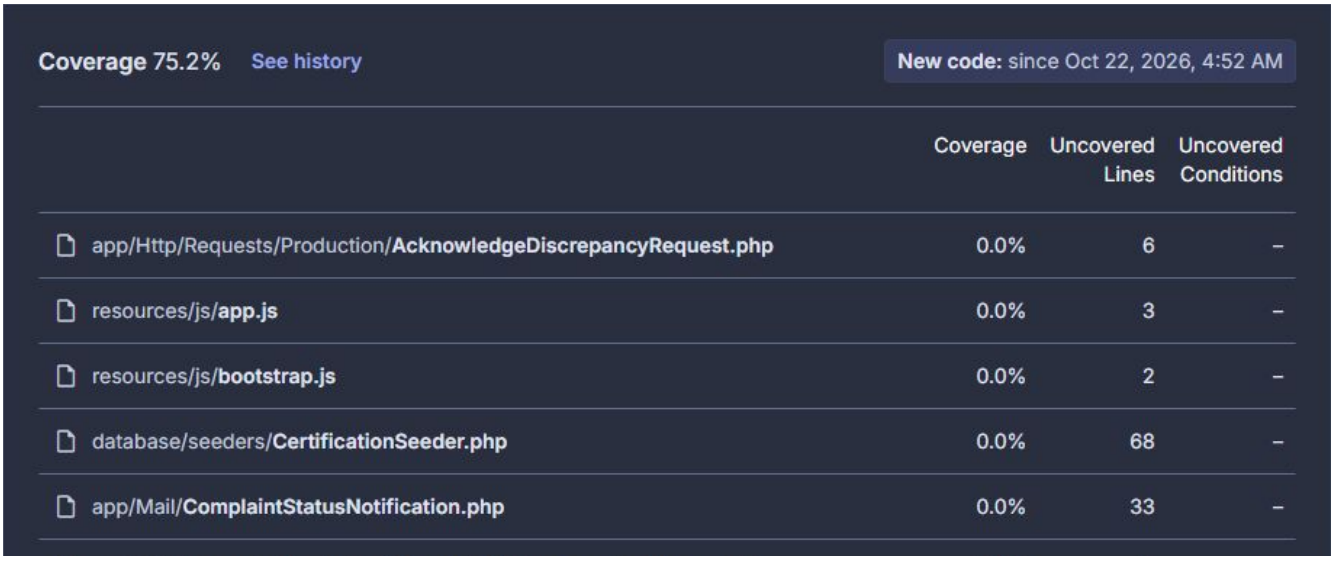


Figure 2. Détail des fichiers : des branches et traitements restent non exercés. Un score de 0 % signale un manque de test ; il ne prouve pas qu'un écran ne fonctionne pas.

Consulter les fichiers et les lignes non couvertes

Pourquoi une ancienne version indiquait 68 %

Le rapport Clover antérieur ne collectait pas **database/**. Le commit **5e84736** a ajouté ce dossier au périmètre de collecte. **68 % et 75,2 % ne sont donc pas directement comparables.** Ce changement de mesure ne démontre pas, à lui seul, une amélioration du code.

Périmètre actuel : code applicatif et configuration, routes, ressources et code de base de données. Les factories de test sont exclues par SonarQube ; leurs 17 chemins présents dans Clover expliquent l'avertissement d'import.

05 SonarQube : résultats et conditions

L'analyse porte sur le projet. Le Quality Gate applique ses conditions au code ajouté ou modifié depuis une référence fixe.



Figure 3. Le gate est vert ; le tableau de bord conserve néanmoins **27 constats globaux**. Le statut vert ne signifie pas « aucun problème dans le projet ».

Ouvrir le tableau de bord SonarQube

Mesure	Projet complet	Condition du gate / changements
Couverture	75,2 %	100 % ; seuil minimal : 80 %.
Duplication	2,0 %	0 % ; seuil maximal : 3 %.
Fiabilité / maintenabilité	20 / 7 constats	Ratings A / A requis et obtenus.
Sécurité	0 vulnérabilité détectée	Rating A ; hotspots revus : 100 %.

Une référence fixe, pas un déplacement après chaque push

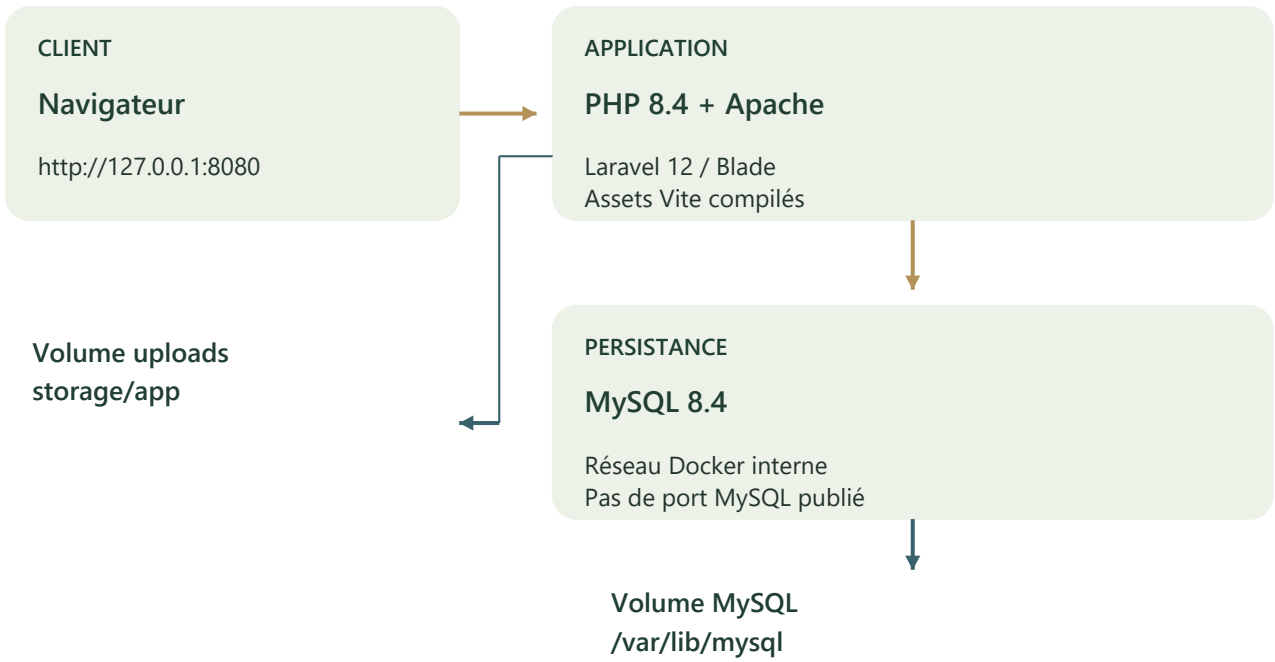
La période « nouveau code » compare les changements depuis **284042c**, version intégrée antérieure au travail DevOps. La couverture **100 %** de ce périmètre restreint ne remplace pas la couverture globale **75,2 %**. Les seuils n'ont pas été abaissés.

Limites et suite : examiner les 20 constats de fiabilité et les 7 de maintenabilité, puis ajouter des tests sur les lignes et scénarios manquants. Zéro vulnérabilité détectée par les règles actives n'est pas une garantie de sécurité absolue.

Conditions du gate et définition du périmètre

06 Une architecture Docker reproductible

L'image contient le code et ses dépendances. Les données et uploads persistent dans des volumes séparés.



Élément	Rôle dans la livraison
Build multi-stage	Node compile les assets ; PHP/Apache reçoit les dépendances Composer de production et l'application.
Démarrage	Dossiers, permissions, lien public et caches Laravel, puis Apache. Les migrations restent une commande explicite.
Persistance	Volumes MySQL et uploads conservés lors de la recréation de l'application.
Exposition	Seul public/ est servi ; .env et les fichiers privés restent hors du web.

Sources figées : Dockerfile · compose.yaml · démarrage

07 Le déploiement local est vérifié

Deux preuves complémentaires : l'application locale accessible et la vérification automatique de l'image de livraison.

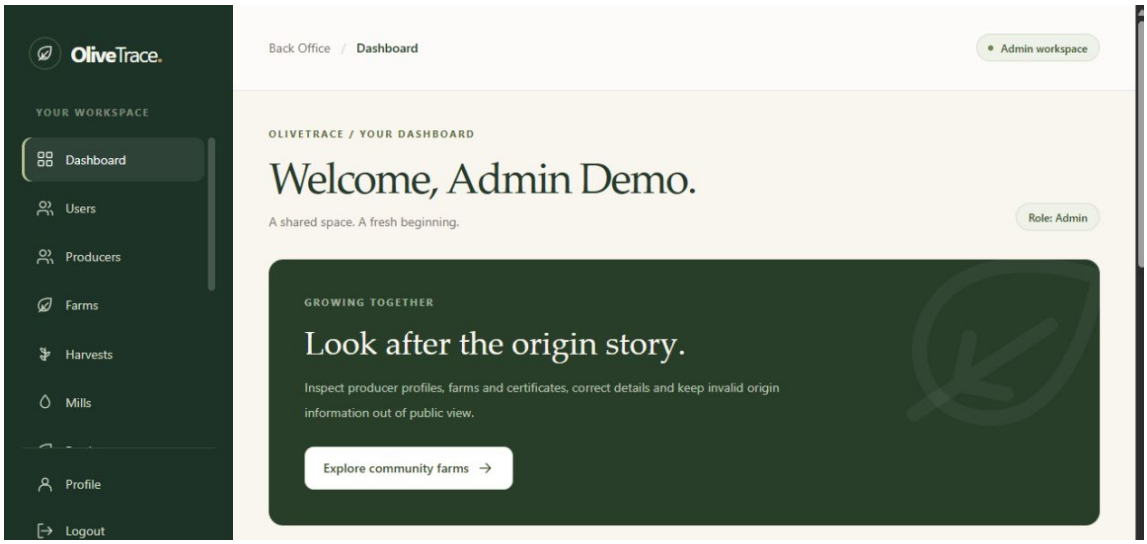


Figure 4. Capture locale du 22 octobre : session administrateur et tableau de bord sur **http://127.0.0.1:8080**. Cette capture du socle Docker précède le correctif Gemini bc96add.

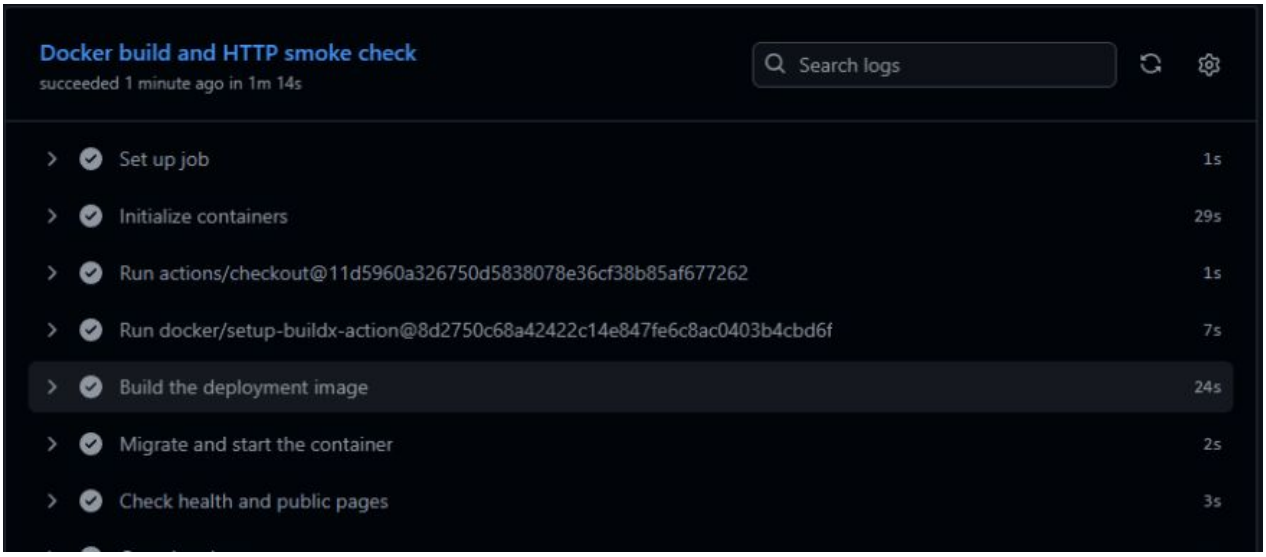


Figure 5. Image du commit **bc96add** vérifiée par la CI : build, migration, démarrage, santé et contrôles des pages publiques réussis.

Ouvrir le job Docker et ses commandes

Contrôles : /up, /, /login et /catalog → 200 ; /.env et un chemin de preuve privée → 404. La conservation des uploads après recréation a été vérifiée lors du contrôle local.

Deux environnements : Docker sur :8080 avec sa propre base ; démonstrations complètes, IA et SMTP sur le serveur natif :8000. Les secrets externes ne sont pas transmis par Compose.

08 Reproduire le déploiement local

Prérequis : Git, Docker Desktop démarré en mode conteneurs Linux, Docker Compose v2, accès Internet et port 8080 libre.

1 · Cloner, construire et préparer les variables privées

```
git clone https://github.com/SofieneZayati/OliveTrace.git
cd OliveTrace
docker build -t olivetrace:local .
docker run --rm --entrypoint php olivetrace:local artisan key:generate --show
Copy-Item docker/variables.example .env.docker
notepad .env.docker
```

Dans **.env.docker**, renseigner DOCKER_APP_KEY avec la clé générée, puis choisir DOCKER_DB_PASSWORD et DOCKER_ROOT_PASSWORD. Conserver ces valeurs privées et stables. Ne pas remplacer un fichier déjà configuré.

2 · Démarrer MySQL, migrer, puis démarrer l'application

```
docker compose --env-file .env.docker up -d mysql
docker compose --env-file .env.docker run --rm app php artisan migrate --force
docker compose --env-file .env.docker up -d app
docker compose --env-file .env.docker ps
```

Attendu : migrations réussies, MySQL **healthy** et application démarrée. Ouvrir **http://localhost:8080**. La base neuve est vide ; on peut tester l'inscription et la connexion sans importer de données privées.

3 · Vérifier les réponses HTTP

```
foreach ($path in @('/up','/','/login','/catalog','/.env')) {
  $url = 'http://localhost:8080' + $path
  "$path : $(curl.exe -s -o NUL -w '%{http_code}' $url)"
}
```

Attendu : 200 pour les quatre premières URL, 404 pour /.env. Les données de démonstration sont facultatives ; la commande dédiée et son périmètre local figurent dans l'annexe technique du dépôt.

Diagnostic / arrêt : `docker compose --env-file .env.docker logs --tail=50 app`
Puis `docker compose --env-file .env.docker stop`. Les volumes sont conservés ; ne pas les supprimer pour une simple mise à jour.

09 Railway : publication non validée

Le dépôt est prêt à être déployé. Aucun service public ni domaine HTTPS OliveTrace n'a encore été vérifié.

CD PUBLIC INCOMPLET

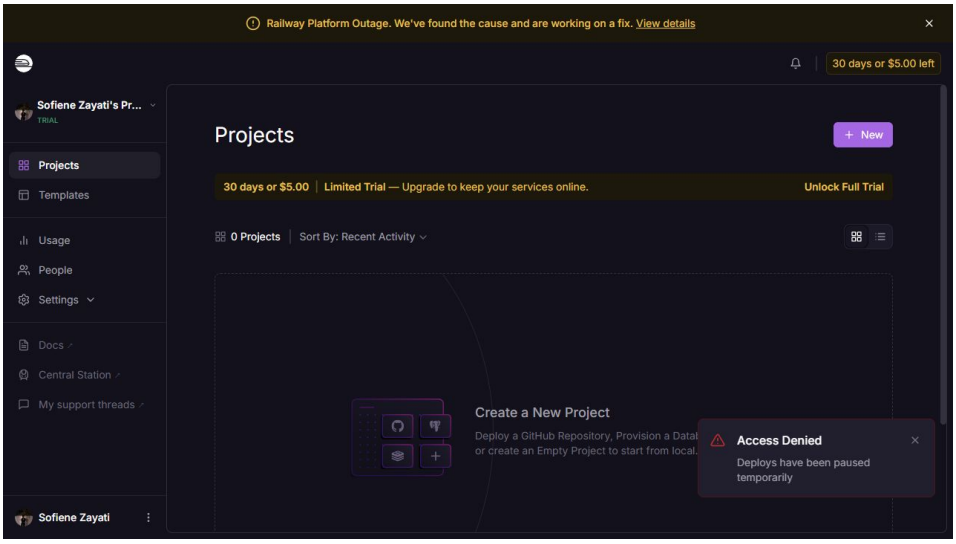


Figure 6. Blocage constaté : « Deploys have been paused temporarily ». Compte en Limited Trial ; aucun service OliveTrace créé pendant cette préparation.

État officiel de l'incident Railway

Point vérifié le 22 octobre : la mise à jour fournisseur de **10 h 15 UTC / 11 h 15 Tunis** maintient les offres Trial en attente de rétablissement. Ce blocage explique l'absence de publication ; il ne constitue pas une preuve de CD réalisé.

État	Élément
Préparé dans Git	Dockerfile, railway.json, exemple de variables privées, migration avant démarrage et santé /up.
À configurer dans Railway	Application reliée à main, MySQL et volumes persistants, variables privées, option Wait for CI.
À démontrer en public	Push accepté après CI, première release, domaine HTTPS, santé, logs et conservation des données.

Chaîne cible : main → CI verte → Railway / Wait for CI → build Docker → migration → démarrage → /up sain → HTTPS. **Cette chaîne n'a pas encore été exécutée de bout en bout.**

10 Exploitation, sécurité et reprise

Un conteneur sain doit aussi pouvoir être observé, mis à jour et rétabli sans perdre ses données.

Besoin	Mise en place / procédure
Santé de Laravel	/up vérifie Laravel et une requête SQL ; base indisponible → statut 500 sans détails de connexion.
Santé de MySQL	Health check SELECT 1 avant le démarrage de l'application dans Compose.
Logs	Apache et Laravel vers stdout/stderr ; consultation avec docker compose logs.
Persistance	Volumes distincts pour MySQL et storage/app. La recréation de l'application préserve les uploads.
Secrets	Variables et clés hors Git ; debug désactivé dans le conteneur ; secrets CI chiffrés dans GitHub.
Avant une migration risquée	Sauvegarde MySQL et uploads ; conservation de APP_KEY ; migrations compatibles privilégiées.

Retour à une version stable

- **Diagnostiquer :** lire les logs et consulter /up ; arrêter la promotion de la version défaillante.
- **Revenir au code validé :** redéployer l'image connue ou annuler le changement par un commit Git.
- **Contrôler les données :** un retour au code n'annule pas les migrations. Une restauration exige une sauvegarde et un contrôle adapté.

Limites opérationnelles

Les logs et checks de santé sont présents. Il n'y a pas de supervision continue avec alertes, ni de Grafana / Prometheus installé. La restauration complète reste une procédure à exercer.

Maintenance CI : renouveler le secret privé SONAR_TOKEN avant le 21 novembre 2026. Aucun membre n'a besoin de ce secret pour lancer l'application localement.

11 Bilan et préparation de la validation

Le processus demandé est documenté ; les résultats vérifiés et le travail restant sont distingués ci-dessous.

Objectif	État constaté
Collaboration et intégration Git	Réalisées : dépôt partagé, branches, historique et PR consultables.
Intégration continue	Réalisée : 244 tests / 1 762 assertions sur chaque base, build, qualité et preuves conservées.
Packaging et déploiement local	Réalisés : image Docker, MySQL, migrations, contrôles HTTP et volumes persistants.
Déploiement continu public	Non validé : configuration préparée dans le dépôt ; services Railway et première release à finaliser.

Conclusion

OliveTrace dispose d'une **CI opérationnelle** et d'un **déploiement local reproductible**. Il ne dispose pas encore d'une démonstration de **CD public de bout en bout**. L'achèvement de cette étape nécessite une publication réellement vérifiée, pas seulement un diagramme ou un fichier de configuration.

Prochaines actions, dans l'ordre

- **1.** Après rétablissement Railway : configurer les services, variables, volumes et l'attente de la CI ; publier une release.
- **2.** Vérifier HTTPS, /up, authentification, données et redéploiement après un changement ; conserver la preuve et l'URL.
- **3.** Examiner les constats SonarQube, enrichir les tests manquants et exercer une restauration.

Parcours de présentation conseillé

Historique / PR → pipeline GitHub Actions → rapports et couverture → Quality Gate → Dockerfile / Compose → application sur :8080 et HTTP → état Railway et limites. Les scénarios métier et l'IA réelle sont montrés séparément sur :8000.

Sources et preuves consultables

Dépôt : révision bc96add · CI #40 et artefacts · SonarQube
Workflow CI · Configuration Railway · Incident fournisseur
Laravel : déploiement · SonarQube : Quality Gate et périmètre